



**JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY**

SCHOOL OF OPEN, DISTANCE AND eLEARNING

P.O. Box 62000, 00200

Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

BIT 2114 Object Oriented Analysis and Design

LAST REVISION ON April 28, 2014





This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



LESSON 10

Production of Specification Document

Learning outcomes

Upon completing this topic, you should be able to:

1. Explain the goals of UML
2. Describe several UML software packages and their capabilities.
3. Translate the specification document into code using the chosen programming language.



10.1. Architecture and the UML

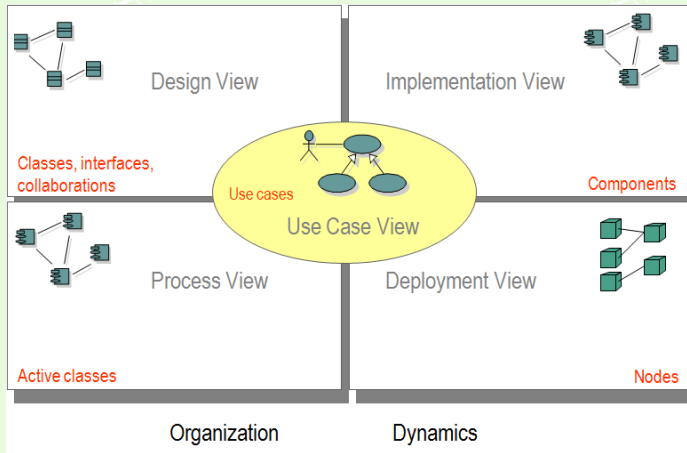


Figure 10.1: Architecture and the UML



10.1.1. Goals of the UML

The primary design goals of the UML are as follows:

1. Provide users with a ready-to-use, expressive visual modeling language to develop and exchange meaningful models.
2. Provide extensibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development processes.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of the OO tools market.
6. Support higher-level development concepts such as collaborations, frameworks, patterns, and components.



7. Integrate best practices.

Provide users with a ready-to-use, expressive visual modeling language to develop and exchange meaningful models

It is important that the OOAD standard supports a modeling language that can be used "out of the box" to do normal general-purpose modeling tasks. If the standard merely provides a meta-meta-description that requires tailoring to a particular set of modeling concepts, then it will not achieve the purpose of allowing users to exchange models without losing information or without imposing excessive work to map their models to a very abstract form.

The UML consolidates a set of core modeling concepts that



BIT 2114 Object Oriented Analysis and Design

are generally accepted across many current methods and modeling tools. These concepts are needed in many or most large applications, although not every concept is needed in every part of every application. Specifying a meta-meta-level format for the concepts is not sufficient for model users, because the concepts must be made concrete for real modeling to occur.

If the concepts in different application areas were substantially different, then such an approach might work, but the core concepts needed by most application areas are similar and should be supported directly by the standard without the need for another layer.



Provide extensibility and specialization mechanisms to extend the core concepts

OMG expects that the UML will be tailored as new needs are discovered and for specific domains. At the same time, we do not want to force the common core concepts to be redefined or re-implemented for each tailored area.

Therefore, we believe that the extension mechanisms should support deviations from the common case, rather than being required to implement the core OOA&D concepts themselves. The core concepts should not be changed more than necessary.

Users need to be able to:

- build models using core concepts without using extension mechanisms for most normal applications,



BIT 2114 Object Oriented Analysis and Design

- add new concepts and notations for issues not covered by the core,
- choose among variant interpretations of existing concepts, when there is no clear
- consensus, and
- specialize the concepts, notations, and constraints for particular application domains.

Be independent of particular programming languages and development processes

The UML must and can support all reasonable programming languages. It also must and can support various methods and processes of building models.



BIT 2114 Object Oriented Analysis and Design

The UML can support multiple programming languages and development methods without excessive difficulty.

Provide a formal basis for understanding the modeling language

Because users will use formality to help understand the language, it must be both precise and approachable; a lack of either dimension damages its usefulness. The formalisms must not require excessive levels of indirection or layering, use of low-level mathematical notations distant from the modeling domain, such as set-theoretic notation, or operational definitions that are equivalent to programming an implementation.

The UML provides a formal definition of the static format of the model using a metamodel expressed in UML class dia-



BIT 2114 Object Oriented Analysis and Design

grams. This is a popular and widely accepted formal approach for specifying the format of a model and directly leads to the implementation of interchange formats. UML expresses well-formedness constraints in precise natural language plus Object Constraint Language expressions.

UML expresses the operational meaning of most constructs in precise natural language. The fully formal approach taken to specify languages such as Algol-68 was not approachable enough for most practical usage.

Encourage the growth of the OO tools market

By enabling vendors to support a standard modeling language used by most users and tools, the industry benefits. While vendors still can add value in their tool implementations, enabling



BIT 2114 Object Oriented Analysis and Design

interoperability is essential. Interoperability requires that models can be exchanged among users and tools without loss of information.

This can only occur if the tools agree on the format and meaning of all of the relevant concepts. Using a higher meta-level is no solution unless the mapping to the user-level concepts is included in the standard.

Support higher-level development concepts such as collaborations, frameworks, patterns, and components

Clearly defined semantics of these concepts is essential to reap the full benefit of OO and reuse. Defining these within the holistic context of a modeling language is a unique contribution of the UML.



Integrate best practices

A key motivation behind the development of the UML has been to integrate the best practices in the industry, encompassing widely varying views based on levels of abstraction, domains, architectures, life cycle stages, implementation technologies, etc. The UML is indeed such an integration of best practices.

10.1.2. Object-Oriented Developing Process

UML is only a notation, not a method. We need an object-oriented developing process.

Famous developing process

- Rational Unified Process
- OMT,



- Booch Method,
- XP, ...

10.1.3. Steps Used in UML

The steps used in UML are:

- Define the use case model
- Define the object model
- Continue UML diagramming to model the system during the systems analysis phase
- Begin system design by refining UML diagrams and using them to derive classes and their properties and methods



10.1.4. Difficulties of Object-Oriented Design

Object Oriented Design

- find pertinent objects,
- factor them into classes at the right granularity,
- define class interfaces and inheritance hierarchies, and
- establish key relationships among them

The design should be specific to the problem at hand but also general enough to address future problems and requirements. Also want to avoid redesign, or at least minimize it . A reusable and flexible design is difficult if not impossible to get “right” the first time.



10.2. From UML to the Unified Process

UML as a model can't work in isolation. Large scale system design/development involves:

- Team-Oriented Efforts.
- Software Architectural Design
- System Design, Implementation, Integration

The Unified Process by Rational is:

- Iterative and Incremental
- Use Case Driven
- Architecture-Centric



10.2.1. Creating the Unified Process

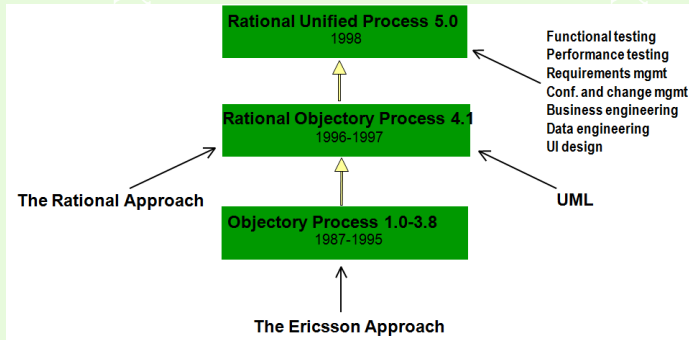


Figure 10.2: Creating the Unified Process

10.2.2. What Is a Process?

It defines who is doing what, when to do it, and how to reach a certain goal.



10.2.3. Lifecycle Phases

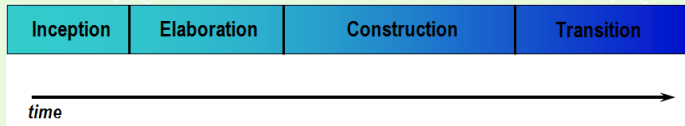


Figure 10.3: Unified Process Phases

- Inception: Define the scope of the project /develop business case
- Elaboration: Plan project, specify features, and baseline the architecture
- Construction: Build the product
- Transition: Transition the product to its users

10.2.4. Unified Process Structure Iterations and Workflow

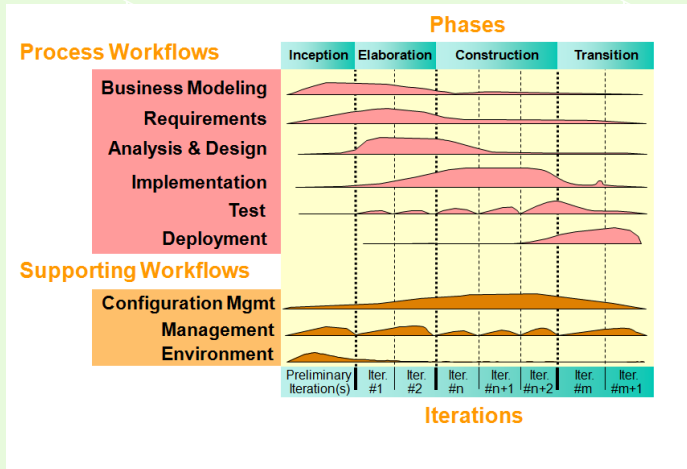


Figure 10.4: Unified Process Workflows

10.2.5. Workflows and Models

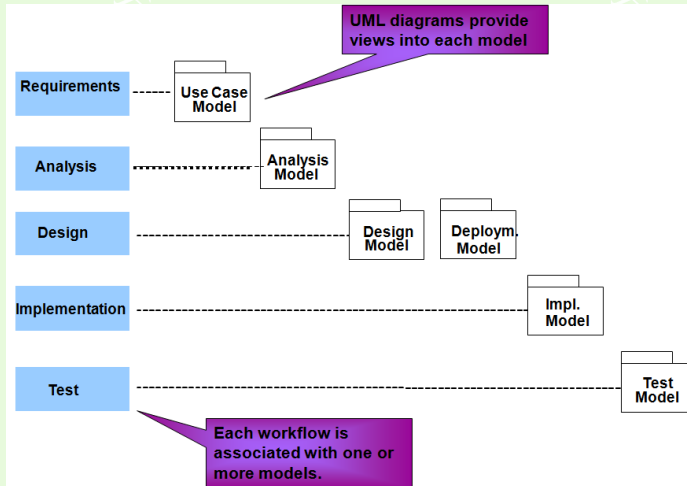


Figure 10.5: Unified Process Workflows and Models

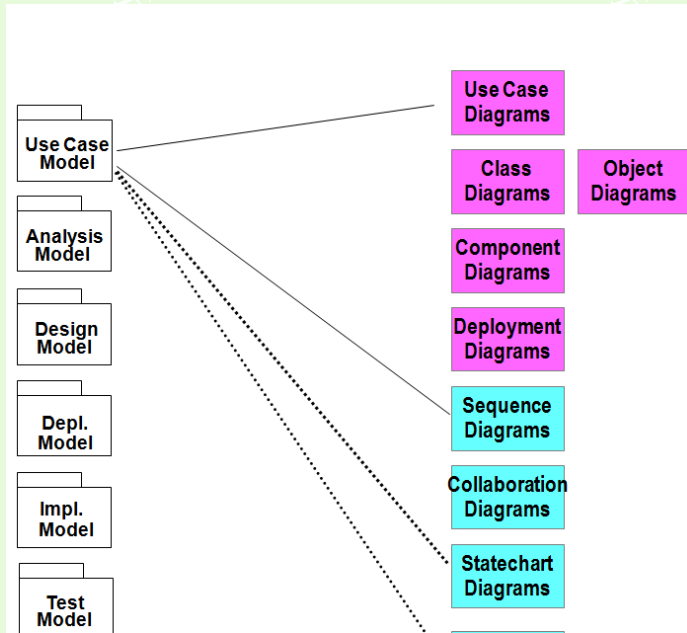


JKUAT SODeL
©2014





10.2.6. Use Case Model





JKUAT SODeL
©2014

Figure 10.6: Use Case Model



10.2.7. Analysis and Design Model

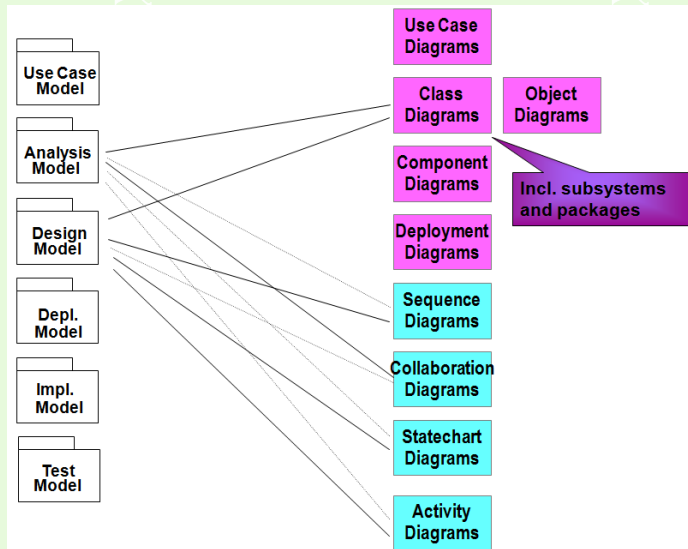


Figure 10.7: Analysis and Design Model



10.2.8. Deployment and Implementation Model

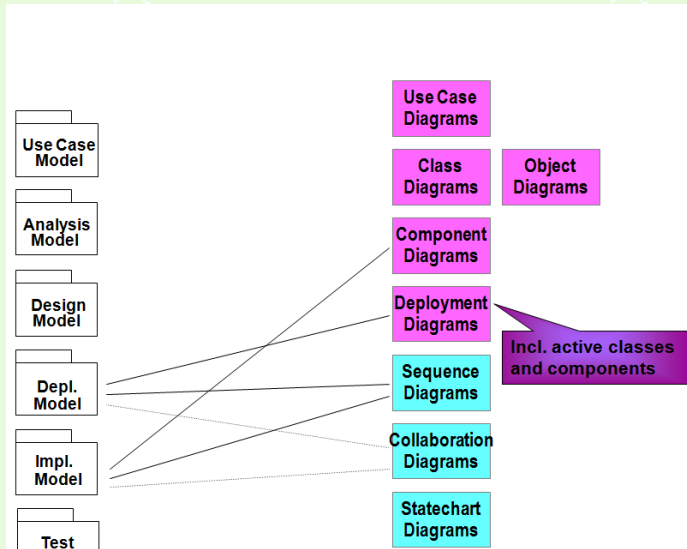


Figure 10.8: Deployment and Implementation Model



JKUAT SODeL
©2014





10.2.9. Test Model

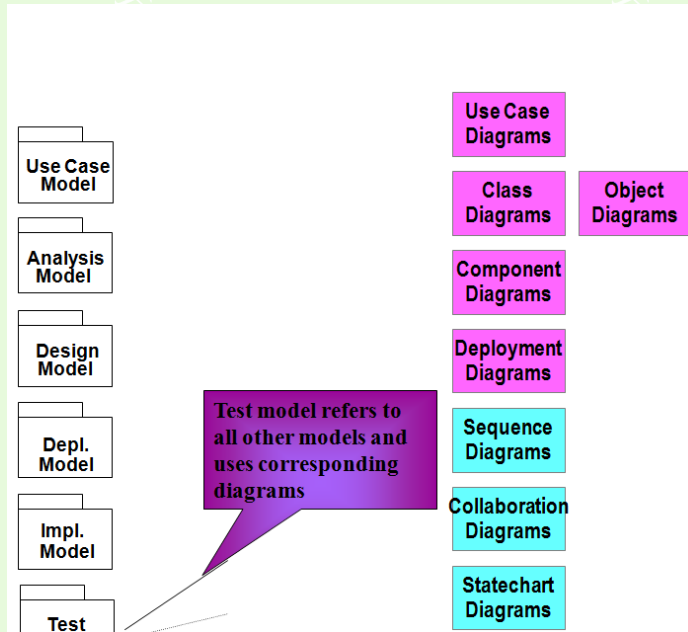




Figure 10.9: Test Model

10.2.10. Use Case Driven

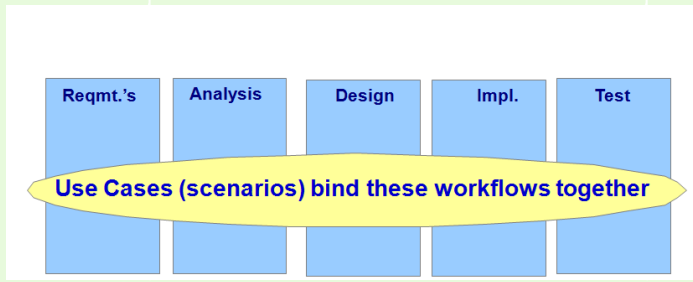


Figure 10.9: Use Case Driven Development

Use Cases Drive Iterations

Drive a number of development activities

- Creation and validation of the system's architecture



- Definition of test cases and procedures
- Planning of iterations
- Creation of user documentation
- Deployment of system

Synchronize the content of different models

10.3. Production of Specification Document

10.3.1. Document Outline

1. Introduction

The document begins with an introductory description of the desired system. Except as noted below, the system is described in present tense, third person, active voice. The purpose of using present tense is to render this document historically relevant



BIT 2114 Object Oriented Analysis and Design

throughout the lifetime of the system. That is, the requirements specification is a permanent part of the system documentation, before the system is implemented, during implementation, and afterward. Later, the system is referred to in future tense in order to distinguish between the state of operations before and after system installation.

1. Problem Statement

The problem statement is a succinct presentation of the specific problem(s) to be solved and the needs to be met by the system.

1. System Personnel

The description of system personnel defines all of the people involved with the system and its development. Personnel categories include some or all of the following:

- system end users



BIT 2114 Object Oriented Analysis and Design

- paying customers
- project managers
- domain experts
- system analysts
- system developers

Membership among these groups may overlap.

1. Operational Setting

The operational setting is the environment in which the system is to be used. The discussion includes how operations are conducted in the environment before and after the system is installed.

The following questions are addressed in the discussion of the operational setting:



BIT 2114 Object Oriented Analysis and Design

- What computer-based support is in use prior to installation of the new system?
- Does the new system need to interface with existing system(s) or are the existing system(s) to be replaced entirely?

1. Impact Analysis

The impact analysis assesses the impacts of the system within its operational setting. Both positive impacts (e.g., increased productivity, higher product sales) as well as negative impacts (e.g., job displacement, potential negative legal impacts) are addressed.

1. Related Systems



BIT 2114 Object Oriented Analysis and Design

Related systems are those computer systems in use elsewhere that provide functionality similar or related to the functionality of the system being specified here.

Both commercially available and public domain systems are considered and the following issues are addressed:

- What is good about the related system, i.e., what features it has that should be included in the system being proposed here.
- What is bad about the related system, i.e., what features should not be included in the proposed system, or what features should be included but done in a different or better way.
- What is missing, i.e., what new features should be included in the proposed system that are not found in the related

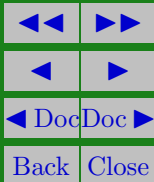


system

- Any other noteworthy features.

10.4. Summary

By now you should have some basics in an OOP programming language e.g Java or C++. Although it is not our aim to write complete Java codes in this unit, you should be able to translate UML diagrams to java code and even Java code to UML diagrams.





Learning Activity I

Define a Java program to implement the following UML diagram. Note that the types have not been indicated in the diagram but you should indicate them in the class definition.

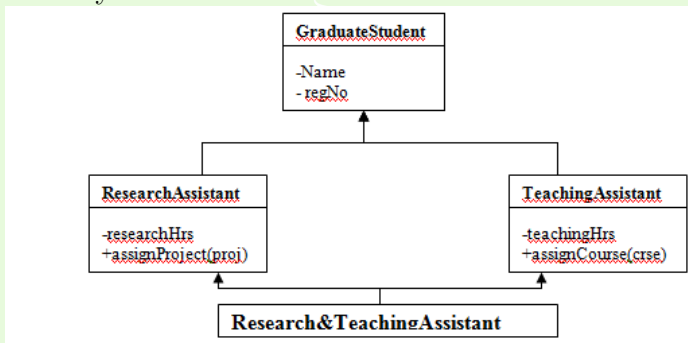


Figure 10.10: UML Specification



Learning Activity II

Below is a JAVA program which implements an Employee system definition. Deduce a UML diagram from the code, clearly giving the attributes and types of each class.

```
public class Employee {  
    private String empName; private Int EmpNumber; private  
String Address; private String DateHired;  
    Employee(String en, Int ENo, String Ad, String dh)  
    { empName = en; EmpNumber = ENo; Address = Ad; Date-  
Hired = dh; }  
    void getName() { }  
    void getNumber() {}  
    void getDateHired() {}  
    void getAddress() {} }
```



BIT 2114 Object Oriented Analysis and Design

```
Class HourlyEmployee extends Employee {  
    private double hourlyRate; private double hours;  
    HourlyEmployee(String en, Int ENo, String Ad, String dh,  
double hR, double Hr) {  
        Super(en, ENo, Ad, dh); hourlyRate = hR; hours = Hr; }  
    Double ComputeWages() {  
        return hourlyRate * hours; }  
    }  
Class SalariedEmployee extends Employee {  
    private double AnnualSalary; SalariedEmployee(String en,  
Int ENo, String Ad, String dh, double As) {  
        Super(en, ENo, Ad, dh); AnnualSalary = As; }  
    void setAnnualSalary() { }  
    }
```



BIT 2114 Object Oriented Analysis and Design

```
class Consultant extends Employee {  
    private String ContractNumber; private double billingRate;  
    private double hours;  
    Consultant(String en, Int ENo, String Ad, String dh, String  
    CN, double Br, double hr){  
        Super(en, ENo, Ad, dh); billingRate= Br; hours = hr; }  
    double ComputeFee() {  
        return billingRate * hours;  
    }  
}
```

Learning Activity III

A bank is finally ready to create a new fully-computerized management system! The system to be developed should enable



BIT 2114 Object Oriented Analysis and Design

users to create bank accounts. Bank accounts maintain their own balance. Customers should be able to deposit and withdraw money to/from their accounts, by visiting a bank teller in one of the branches. Customers should also be able to transfer funds from one bank account to another (in other words, the money should be withdrawn from the source account, and deposited into the destination account). The Bank manager should also calculate the interest accumulated each month for each account, and deposit that amount into the account. Also, the bank manager should also calculate the monthly charges each month (for each account), and withdraw that amount from the account.

Use Case Determination

1. List the three actors present in the Bank's system.



BIT 2114 Object Oriented Analysis and Design

2. List the six use cases present in the Bank's system. Hint: Do not list use cases not mentioned in the specification.
3. Draw the completed Use Case Diagram showing all 3 actors and all 6 use cases, as well as any relationships that might exist.

Class Identification

1. List the entity classes present in the Bank's System. Hint: Entity classes are classes that represent data or tangible items that the system would need to represent in some way.
2. Think of what attributes would be needed to support the Bank's System. Hint: Do not think about attributes not mentioned in the specification.



3. Draw a class diagram containing all classes listed in (1) and all attributes listed in (2), and also include any class relationships that might exist.

Learning Activities

1. Most modern development methods are based on an object-oriented approach to systems development.
 - Discuss why it is important for an object-oriented development method to be architecture-centric?
 - What does it mean for an object-oriented development method to be iterative and incremental?
 - Describe how the Unified Modeling Language (UML) diagrams support the USDP and Object Oriented software development in general



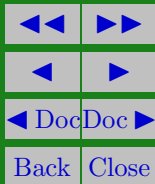
BIT 2114 Object Oriented Analysis and Design

1. Produce a statechart diagram for the Bank Account class described below. Your diagram should include any guard conditions related to transitions between states:

A Bank Account can be in credit, debit or have a balance of zero. The events that affect these states are Charge (when money is paid into the account), Payment (when money is taken out of the account) and Close (when the customer closes the account). When an account is opened it has a balance of zero. From that point the account will move between the states indicated above depending on the value of payments and charges made.

Revision Questions

Example . The Unified software development process identifies four phases for software development. Explain each of the





BIT 2114 Object Oriented Analysis and Design

phases indicating the workflows (activities) associated with each.

Solution: for revision



EXERCISE 1. 🖋 Rational Unified Process (RUP) is a use case driven, architectural centric and iterative and incremental process. Explain the significance of use case driven, architectural centric and iterative and incremental process.

EXERCISE 2. 🖋 The Systems Development Life Cycle (SDLC) has had an enormous influence as a generalized method for developing systems. a) Distinguish between techniques, tools and methodologies in the context of the SDLC. b) What are the strengths and limitations of the traditional SDLC? c) Would you adopt the SDLC for developing a small database application on your stand-alone PC? Would you modify the SDLC in



JKUAT SODeL
©2014

BIT 2114 Object Oriented Analysis and Design

any way? Justify your answer.





References and Additional Reading Materials

1. Richard Lee , William M.Tepfenhart (2000). UML and C++ - A practical guide to object oriented development” , ISBN-13: 978-0130290403
2. John. W Satzinger (2004). Object Oriented Analysis and Design with the Unified Process. Cengage Learning, ISBN-13: 978-0619216436
3. Journal of Science and Technology ISSN 1860-4749